

SENAI SC - JOINVILLE SUL

Engenharia de Software

ATIVIDADE PRÁTICA

Desenvolvimento de Sistema de Gestão de EPIs

Planejamento, Análise e Design de Software - Solução: SafeEPI

Disciplina: Engenharia de Software

Professora: Mestra Tatiana Aparecida de Almeida

Período: 2026

Aluno: Raphael Gouvêa Reche

Matrícula: 26164063

1. APRESENTAÇÃO DA ATIVIDADE

Esta atividade aplica os conceitos de Engenharia de Software nas etapas iniciais do ciclo de vida (Planejamento, Análise e Design). A solução documentada é o SafeEPI, um sistema de gestão de Equipamentos de Proteção Individual que, além de documentado, foi implementado e publicado em nuvem.

Objetivos de aprendizagem:

- Compreender e documentar problemas reais de gestão de processos.
- Aplicar técnicas de levantamento de requisitos funcionais e não funcionais.
- Desenvolver habilidades de análise e definição de necessidades de usuários.
- Criar protótipos de interface seguindo princípios de UX/UI.
- Validar soluções através de feedback estruturado.

2. CICLO DE VIDA DO SOFTWARE

Etapas do ciclo	Trabalhada?	Descrição
Planejamento	SIM	Entender o problema e definir objetivos
Análise	SIM	Levantar e documentar requisitos
Design	SIM	Criar protótipos e arquitetura visual
Implementação	SIM*	Programação do sistema (* feita além do escopo mínimo)
Testes Técnicos	PARCIAL	Validação manual e verificação de regras
Implantação	SIM*	Publicado na Vercel (* além do escopo mínimo)
Manutenção	NÃO	Correções e melhorias contínuas

Observação: o foco da atividade são as três primeiras etapas; a implementação e a publicação foram realizadas como aprofundamento, validando na prática o que foi planejado.

3. ESTUDO DE CASO: SISTEMA DE GESTÃO DE EPIs

A SENAI SC Joinville Sul gerencia EPIs de forma totalmente manual, com planilhas de papel e registros desorganizados, o que compromete a segurança e a eficiência operacional. Os EPIs são essenciais para a segurança de funcionários e visitantes nas dependências e atividades práticas da instituição.

Missão: desenvolver a documentação e o design de um sistema informatizado que resolva os problemas identificados, automatizando o controle de EPIs e melhorando a gestão de segurança da instituição.

4. ETAPA 1 - PLANEJAMENTO

4.1 Contextualização do Problema

Situação atual:

No SENAI SC Joinville Sul, o controle de EPIs é feito de forma manual, com planilhas de papel e registros avulsos. Cada entrega a funcionários e visitantes é anotada à mão, sem um sistema central que relacione o equipamento, a pessoa, a data e o responsável. O estoque é conferido visualmente, sem atualização automática quando um EPI é entregue ou devolvido, e não há acompanhamento das validades (CA e do próprio equipamento).

Principais problemas identificados:

- Registros em papel sujeitos a perda, rasura e extravio.
- Demora e dificuldade para localizar informações.
- Falta de rastreabilidade: não se sabe quem está com cada EPI no momento.
- Estoque sem controle em tempo real, gerando falta ou excesso de itens.
- Risco de segurança por entrega de EPI vencido ou ausência de controle.
- Reposição reativa, sem alerta de estoque mínimo.
- Ausência de relatórios para decisão gerencial.

4.2 Definição do Problema

O controle manual de EPIs no SENAI Joinville Sul, baseado em planilhas de papel, impede a rastreabilidade de quem está com cada equipamento e o controle do estoque e das validades em tempo real, comprometendo a segurança dos usuários e a eficiência operacional.

4.3 Identificação dos Usuários

Perfil	Papel no sistema
Administrador (almoxarife / gestor)	Cadastra EPIs, funcionários e setores; gerencia estoque; aprova solicitações; acessa tudo.
Usuário operacional	Registra entregas e devoluções, solicita EPIs, consulta posse e gera relatórios.
Funcionário / Visitante	Recebe os EPIs e confirma o recibo com assinatura digital (não acessa o sistema).
Gestão / Coordenação	Consome dashboard e relatórios para decisão.

5. ETAPA 2 - ANÁLISE DE REQUISITOS

5.1 Requisitos Funcionais

Código	Descrição do requisito funcional
RF01	O sistema deve permitir o cadastro de EPIs com nome, CA, validade do CA e validade do próprio equipamento.
RF02	O sistema deve permitir o cadastro de funcionários com nome, matrícula, setor e cargo.
RF03	O sistema deve registrar a entrega de EPIs a funcionários, com data, quantidade e assinatura digital, dando baixa automática no estoque.
RF04	O sistema deve registrar a devolução de EPIs, repondo o estoque automaticamente quando o item for reutilizável.
RF05	O sistema deve manter o histórico de movimentações de estoque (entradas, saídas e baixas).

Código	Descrição do requisito funcional
RF06	O sistema deve informar quais EPIs cada funcionário possui no momento (entregue - devolvido).
RF07	O sistema deve emitir alertas de estoque baixo/zerado e de validades próximas do vencimento.
RF08	O sistema deve gerar relatórios de movimentação por período e exportá-los em PDF.
RF09	O sistema deve controlar o acesso por login/senha e por perfil (administrador/usuário).
RF10	O sistema deve permitir a inativação (exclusão lógica) de EPIs e funcionários, preservando o histórico.
RF11	O sistema deve permitir que o usuário solicite EPIs para um funcionário (status pendente); o administrador aprova - gerando a entrega com baixa de estoque - ou rejeita.

5.2 Requisitos Não Funcionais

Código	Descrição do requisito não funcional
RNF01	Segurança: proteger os dados com autenticação e políticas de acesso no banco (RLS), restringindo a escrita a usuários autorizados.
RNF02	Usabilidade: interface responsiva (desktop e mobile) e intuitiva, com navegação por menu lateral.
RNF03	Desempenho: consultas e telas devem carregar em poucos segundos sob uso normal.
RNF04	Disponibilidade: o sistema deve ser acessível via web (nuvem), com backend gerenciado (Supabase).
RNF05	Manutenibilidade: código modular (componentes/views) e banco versionado por migrações SQL.

5.3 Ideação - Funcionalidades do Sistema

Login com perfis; cadastro de EPIs com validade CA/EPI; cadastro de funcionários e setores; controle de estoque (entrada/saída/baixa); baixa protegida por senha; entrega de múltiplos EPIs com assinatura digital; baixa automática de estoque na entrega; devolução com reposição (reutilizável/descarte); solicitação de EPIs com aprovação do administrador (a aprovação gera a entrega e baixa o estoque); consulta de EPIs em posse por funcionário; alertas de validade, estoque mínimo e solicitações pendentes; dashboard com KPIs e gráficos; rankings de funcionários/EPIs; relatórios com filtros e exportação em PDF; inativação lógica; controle de acesso admin/usuário; PDF comercial com números do sistema; diagramas DER/MER acessíveis no dashboard.

6. ETAPA 3 - DESIGN E PROTOTIPAÇÃO

6.1 Prototipação de Telas

Tela 1 - Login: fundo azul com gradiente, logo e nome "SafeEPI", subtítulo; card central com e-mail e senha (com ícones), botão "Acessar Sistema" e mensagem de erro para credenciais inválidas. Layout responsivo.

Tela 2 - Cadastro de EPIs: formulário com nome, nº do CA, validade do CA e a opção "tem validade própria?" (campo de data condicional). Abaixo, tabela dos EPIs com nome, CA, validades, status (válido/vencido) e ações (editar, inativar); busca e opção de mostrar inativos.

Tela 3 - Movimentação (Entrega/Devolução): na entrega, escolhe-se o funcionário e a data, marcam-se vários EPIs (com saldo e validade), informa-se a quantidade e confirma-se a assinatura digital; o estoque

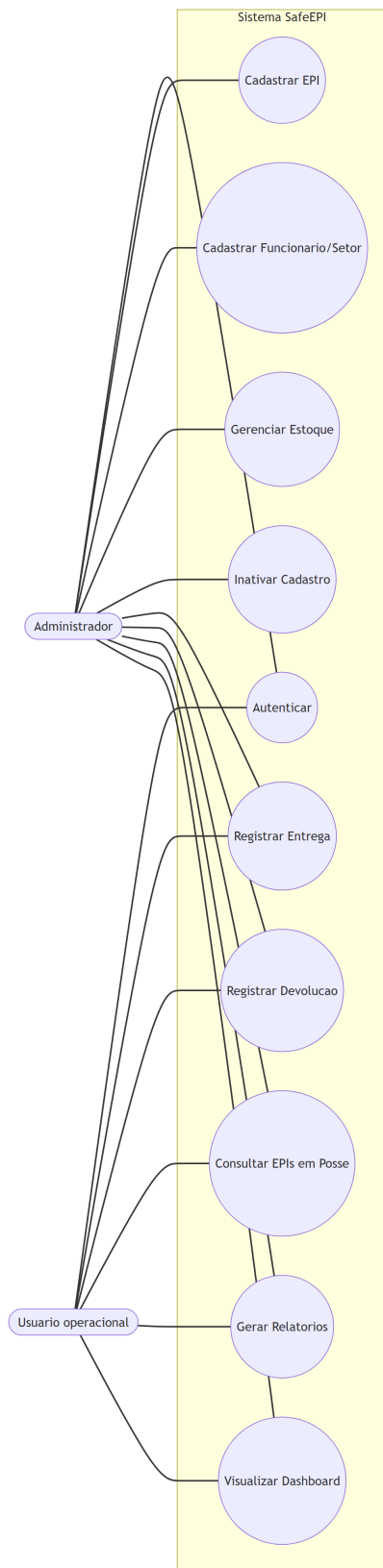
baixa automaticamente. Na devolução, ao escolher o funcionário aparecem só os EPIs em posse; escolhe-se o EPI, a quantidade e a condição (reutilizável -> volta ao estoque; descarte).

Tela 4 - Consultas e Relatórios: filtros por período, funcionário, EPI e CA; tabela com validade, quantidade e assinatura; imprimir e exportar PDF. O Dashboard complementa com KPIs (saldo, posse, vencidos), alertas, gráficos (pizza/barras/linha) e rankings.

Tela 5 - Solicitação de EPIs (nova): o usuário escolhe funcionário, EPI e quantidade e registra uma solicitação (status pendente). O administrador vê a lista, confirma a assinatura digital e clica em "Aprovar" - gerando a entrega e baixando o estoque - ou em "Rejeitar". O Dashboard exibe alerta de solicitações pendentes.

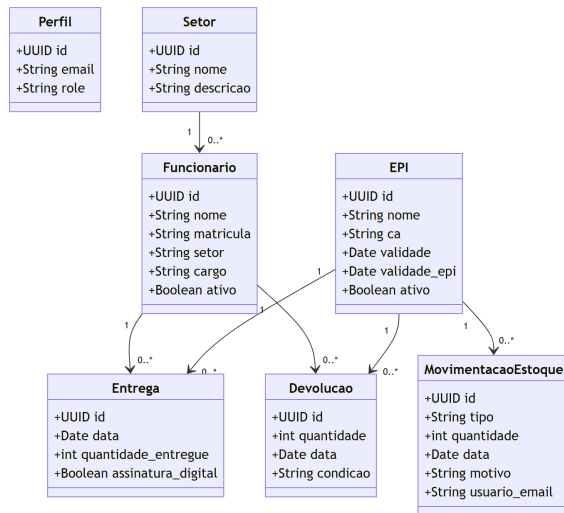
[Anexar os prints reais das telas do sistema rodando em <https://epis-new-bpyu.vercel.app/>]

6.2 Diagrama de Casos de Uso (UML)



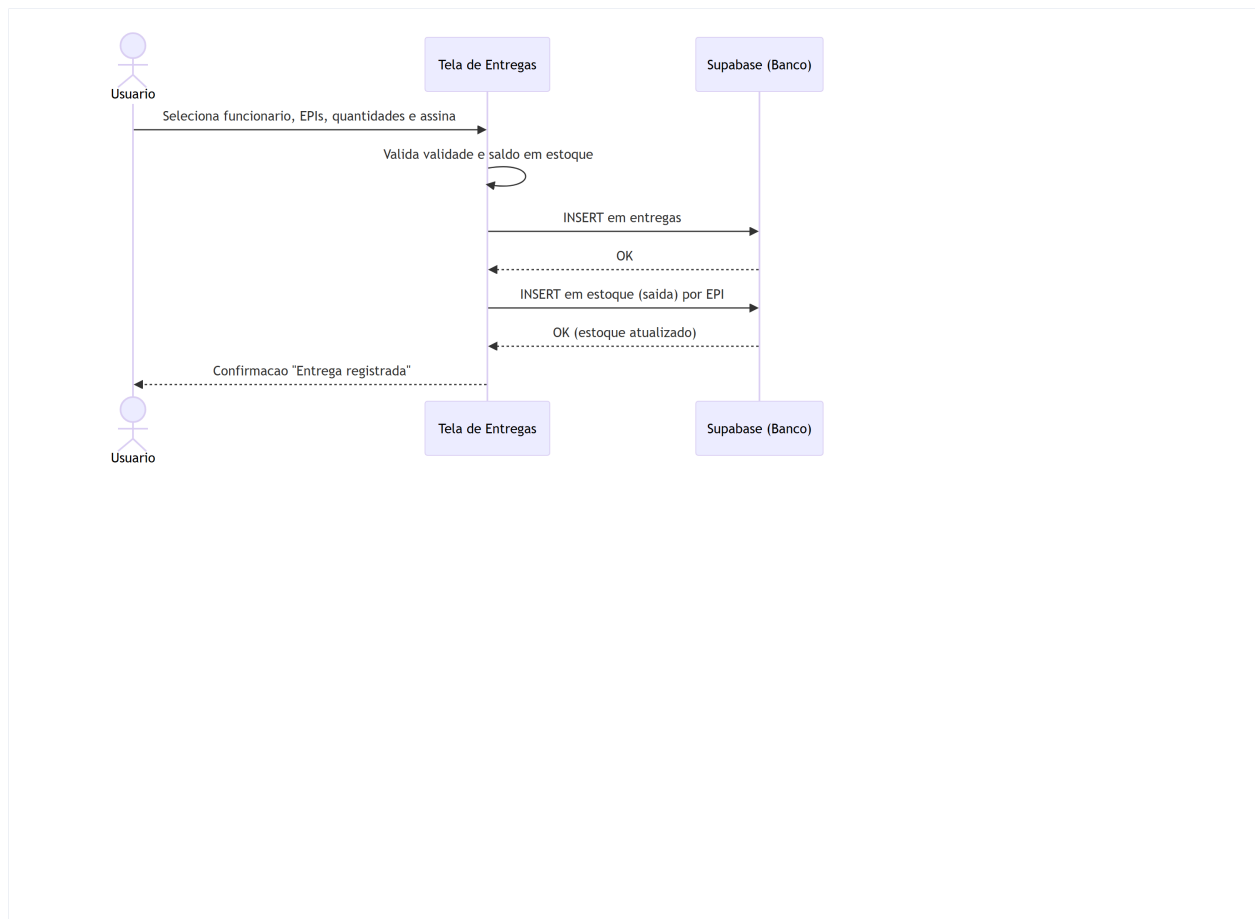
Atores Administrador e Usuário operacional e os casos de uso do SafeEPI.

6.3 Diagrama de Classes (UML)



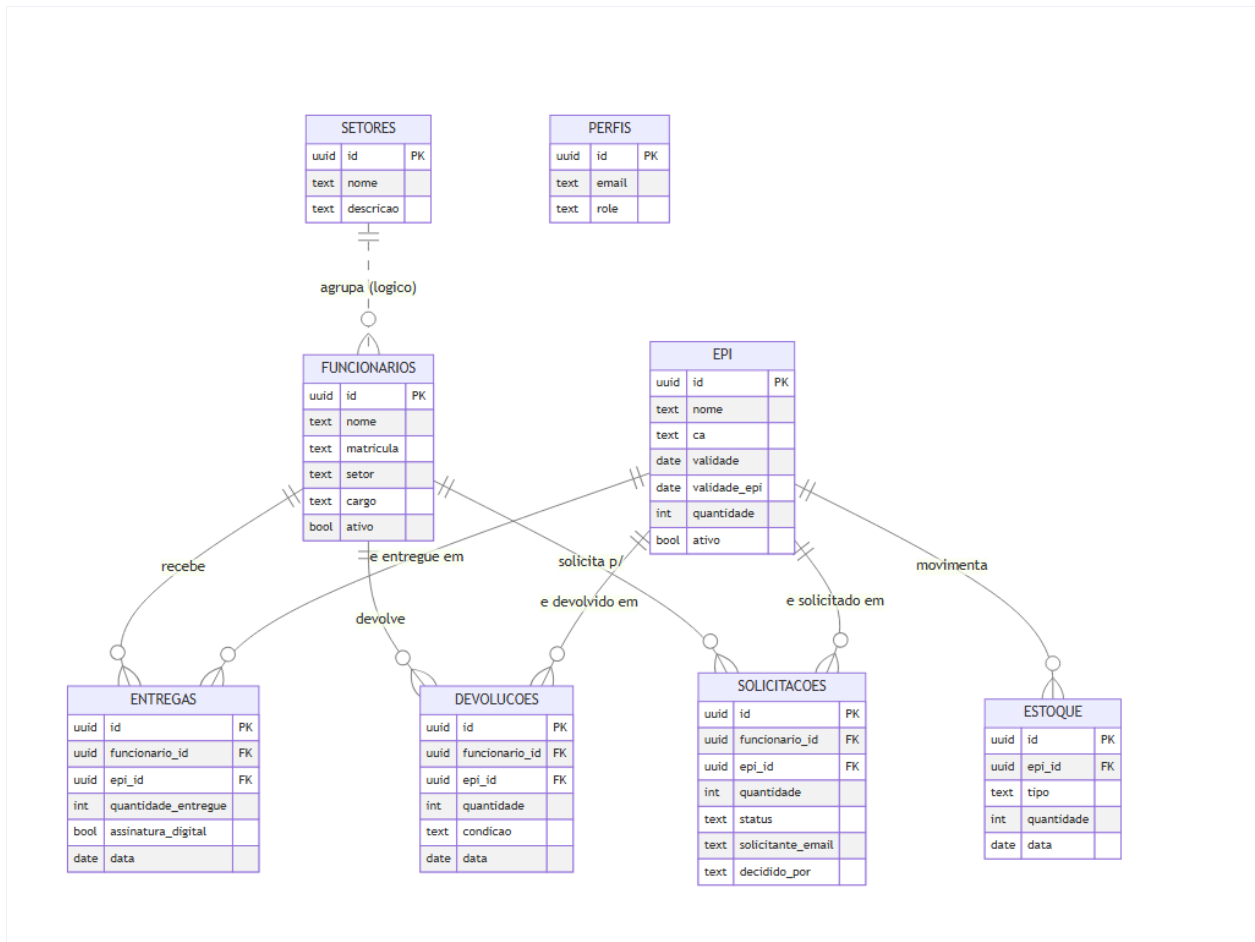
Classes do domínio e seus relacionamentos.

6.4 Diagrama de Sequência (UML) - Registrar Entrega



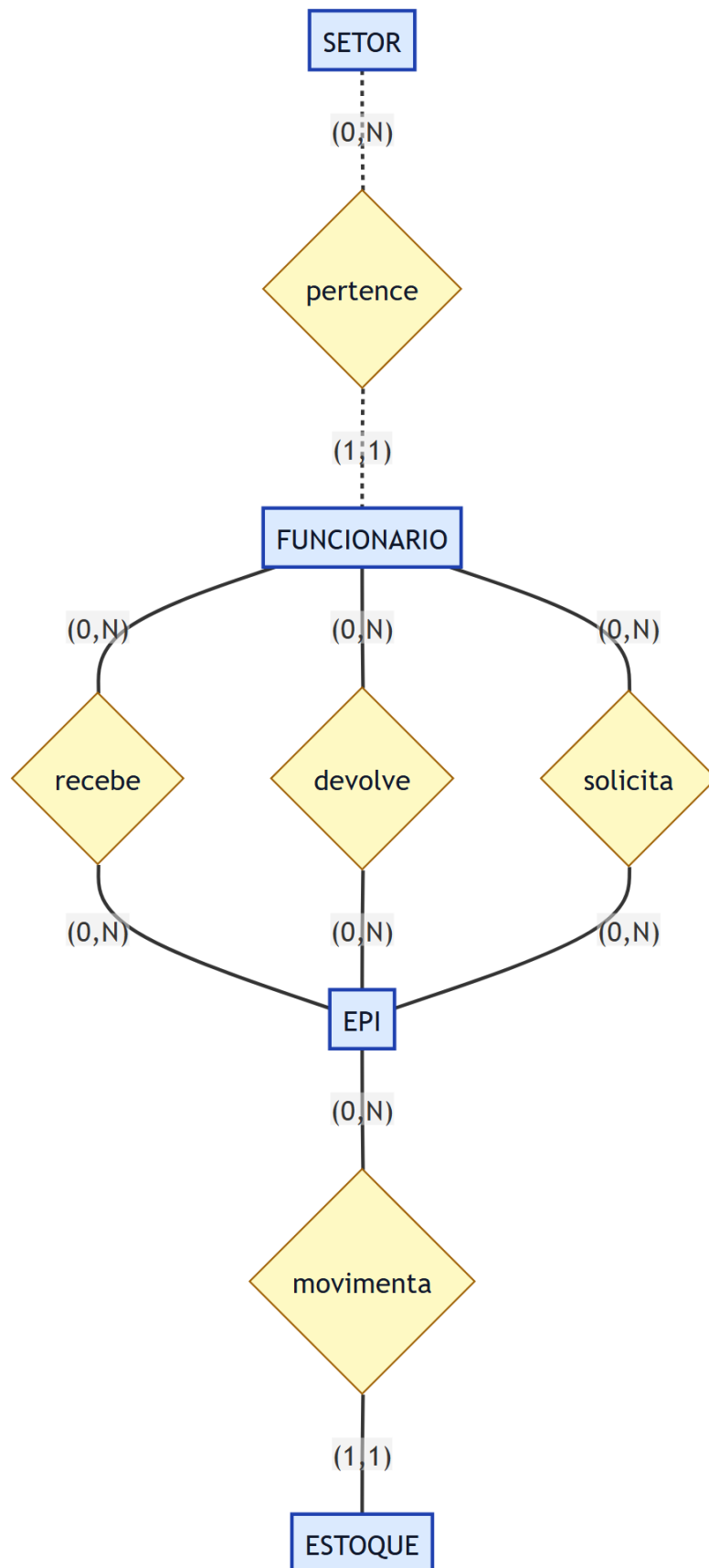
Fluxo de registro de entrega com baixa automática de estoque.

6.5 Diagrama Entidade-Relacionamento (BD)



Modelo de dados (PostgreSQL/Supabase), já com a entidade solicitacoes.

6.6 Modelo Entidade-Relacionamento Conceitual (MER)



Modelo conceitual (Chen) com as relações recebe, devolve, solicita, pertence e movimenta.

7. VALIDAÇÃO - FEEDBACK DO USUÁRIO

Pontos positivos identificados:

- Interface limpa e responsiva.
- Baixa automática de estoque na entrega.
- Alertas de validade, estoque e solicitações pendentes no dashboard.
- Rastreabilidade de quem está com cada EPI.
- Exportação de relatórios em PDF.
- Fluxo de solicitação com aprovação do administrador (pedido -> aprovação -> entrega).

Pontos de melhoria identificados:

- Estoque mínimo configurável por EPI (hoje é um limite fixo).
- Cadastro de visitantes, além de funcionários.
- Notificações por e-mail (vencimento e novas solicitações).
- Leitura por código de barras / QR Code.

8-11. IMPLEMENTAÇÃO E ESTRUTURA DO PROJETO

Após a validação, o projeto avançou para a implementação real, com arquitetura modular e navegação intuitiva, refletindo as melhorias identificadas no protótipo. A organização das pastas (VS Code) facilita a manutenção e o entendimento do fluxo de dados.

```
EPIS_NEW/  
|-- docs/                # Documentação + diagramas (DER/MER, schema-safeepi.sql)  
|-- public/              # Arquivos estáticos  
|-- migracoes.sql        # Migrações do banco (Supabase)  
|-- migracoes_02_inativacao.sql  
|-- migracoes_03_devolucao.sql  
|-- migracoes_04_perfis_rls.sql  
|-- migracoes_05_solicitacoes.sql  
|-- index.html  
|-- package.json  
|-- vite.config.js  
`-- src/  
    |-- App.vue  
    |-- main.js  
    |-- assets/           # Imagens e CSS (logoEPI.jpg, main.css)  
    |-- components/       # appLayout.vue, menu.vue, footer.vue  
    |-- composables/      # useSupabase.js (cliente + sessão + papel)  
    |-- router/           # index.js (rotas + proteção por perfil)  
    |-- utils/            # pdfComercial.js, diagramaImg.js  
    `-- views/            # login, home, dashboard, epi, funcionario,  
                          # setor, estoque, entrega, solicitacao,  
                          # devolucao, posse, relatorio
```

Stack: Vue 3 + Vite (front-end) · Supabase/PostgreSQL (back-end, autenticação e RLS) · Chart.js · jsPDF.

Repositório (GitHub): https://github.com/recheraphael-Re/EPIS_NEW

Aplicação (Deploy): <https://epis-new-bpyu.vercel.app/>

18. CONSIDERAÇÕES FINAIS

Esta atividade mostrou na prática como as etapas iniciais da Engenharia de Software - planejamento, análise e design - sustentam a implementação. Levantar os requisitos e prototipar antes de codificar

evitou retrabalho e deixou claro o que o sistema precisava entregar. A maior dificuldade foi traduzir as regras de negócio (saldo de estoque, "EPis em posse", devolução com reposição e o fluxo de solicitação com aprovação) em um modelo de dados coerente. Acrescentar a solicitação depois da primeira versão reforçou a importância de um bom modelo: bastou criar a tabela solicitacoes e reaproveitar a lógica de entrega já existente. Em um próximo projeto, eu investiria ainda mais cedo nos diagramas UML e na validação com usuários antes de programar.

19. DECLARAÇÃO DE AUTORIA

Declaro que desenvolvi esta atividade de forma individual e autônoma, conforme as orientações da disciplina de Engenharia de Software, respeitando os princípios éticos e acadêmicos.

Nome completo: Raphael Gouvêa Reche

Matrícula: 26164063

Data: / /

Assinatura: _____

20. CHECKLIST DE ENTREGA

- ☒ Descrição da situação atual e problemas identificados
- ☒ Definição clara do problema em uma frase
- ☒ Identificação de todos os perfis de usuários
- ☒ No mínimo 5 requisitos funcionais (11 descritos)
- ☒ No mínimo 3 requisitos não funcionais (5 descritos)
- ☒ Lista de funcionalidades (brainstorming)
- ☒ Descrição das 4+ telas principais (5 descritas)
- ☐ Protótipos visuais anexados (prints reais das telas)
- ☒ Diagrama de Casos de Uso (UML)
- ☒ Diagrama de Classes (UML)
- ☒ Diagrama de Sequência (UML)
- ☒ Diagrama de Entidade-Relacionamento (com solicitacoes)
- ☒ Estrutura de Pastas do Projeto (VS Code)
- ☒ Código e Deploy (links informados)
- ☒ Feedback com pontos positivos e de melhoria
- ☒ Considerações finais reflexivas
- ☐ Declaração de autoria assinada (falta data e assinatura)
- ☒ Documento completo, organizado e revisado